Alpha Quant Model: Python Coding Topics

This document outlines the specific Python coding topics, libraries, and sub-topics used across the three different strategies in the Alpha Model.

1. Alpha Quant V5 PRO (Multifactor Breakout)

The most complex strategy utilizing a mix of statistical and mathematical libraries.

* scipy.stats.linregress & numpy.polyfit (Ordinary Least Squares): Used to calculate the linear regression slope of the stock's closing prices over a 20-day window to determine momentum mathematically.
* pandas Vectorized Rolling Windows: Used extensively for calculating the "Quant Trio":
  - .rolling(window=28).max() and .min() for the Vertical Horizontal Filter (VHF).
  - .rolling(window=14).apply() or native math for the Average True Range (ATR).
  - .rolling(window=50).median() for Bollinger Band Volatility Contraction.
* Statistical Standardization (Z-Scores): Calculating the rolling mean and standard deviation (.std()) of the linear regression slope to rank momentum based on statistical significance (Z > 1.0).
* numpy.where (Conditional Logic): Used to evaluate the 10 simultaneous breakout conditions efficiently across the whole 500-stock universe without slow python for-loops.

2. Monthly Momentum V1 (Top 20 Rotation)

A simpler, cross-sectional ranking strategy heavily reliant on Pandas array manipulations.

* Cross-Sectional Ranking: Using pandas.DataFrame.rank(pct=True, ascending=False) to rank all Nifty 500 stocks based on their 126-day return profile.
* Percentage Returns with shift(): Using (df['Close'] / df['Close'].shift(126)) - 1 to calculate the 6-month momentum while strictly avoiding look-ahead bias.
* Data Masking / Filtering: Applying Boolean masks (df[df['Volume'] > min_vol]) to filter out penny stocks and illiquid counters before the ranking engine runs.
* Periodic Resampling (resample('M')): Handling the monthly rebalancing logic by isolating the last trading day of each month for portfolio rotation.

3. Momentum V2 "God Mode" (Concentrated Leverage)

The most robust strategy, featuring complex robustness testing algorithms and statistical weighting.

* 12M - 1M Momentum Math: Using df['Close'].shift(21) / df['Close'].shift(252) to calculate Jegadeesh-Titman momentum (skipping the most recent month to avoid mean reversion).

Alpha Quant Model: Python Coding Topics

* Inverse Volatility Weighting Models: Using numpy.std() * numpy.sqrt(252) to calculate annualized realized volatility, then weighting the portfolio via (1 / vol) / sum(1 / vol).
* Monte Carlo Simulation Engine (numpy.random):
  - Noise Injection: np.random.normal(loc=0, scale=0.005, size=len(returns)) to simulate slippage and bid-ask spread variations over thousands of runs.
  - Trade Shuffling: np.random.permutation(returns) to randomly re-sequence trades to mathematically prove the strategy's edge isn't reliant on "lucky timing".
* Walk-Forward Validation Engine: Implementing dynamic slicing of pandas dataframes iteratively (train = df[:year], test = df[year:year+1]) to avoid curve-fitting.

4. Shared Data Infrastructure (All Models)

* PyArrow & Apache Parquet (to_parquet / read_parquet): Used for massive data compression. Parquet stores data in columns rather than rows, allowing the script to load 10 years of Nifty 500 data (~1.5 million rows) into memory in fractions of a second.
* Object-Oriented Programming (OOP) & State Management: The PortfolioManager classes use dictionaries and json.dump()/load() to accurately track live positions, stop losses, and target exits across days.
* yfinance API Integration: Used for the daily cron-jobs to connect to Yahoo Finance's REST APIs, downloading day-end Open/High/Low/Close/Volume data to feed the signal engines.